2026-7-7 Linux进程--基础概念

- 1. 从硬件系统到操作系统
  - 1.1 冯·诺依曼体系结构
    - 1.1.1 计算机的基本组成
    - 1.1.2 数据为什么必须先进入内存
    - 1.1.3 从数据流过渡到进程
  - 1.2 操作系统
    - 1.2.1 操作系统的概念
    - 1.2.2 对上提供环境，对下管理资源
    - 1.2.3 “管理”的本质：先描述，再组织
    - 1.2.4 系统调用、库函数与 shell
- 2. Linux 进程基础
  - 2.1 从程序到进程
    - 2.1.1 程序是什么
    - 2.1.2 进程是什么
    - 2.1.3 进程与程序的核心关系
  - 2.2 进程控制块 PCB 与 task_struct
    - 2.2.1 PCB：描述进程的数据结构
    - 2.2.2 task_struct：Linux 的 PCB
    - 2.2.3 task_struct 中的基础属性
    - 2.2.4 内核如何组织进程
  - 2.3 查看进程
    - 2.3.1 通过 /proc 查看进程
    - 2.3.2 通过 ps 和 top 查看进程
    - 2.3.3 通过系统调用获取 PID 和 PPID
  - 2.4 创建进程：fork 初识
    - 2.4.1 fork 做了什么
    - 2.4.2 返回值与父子进程分流
    - 2.4.3 父子进程的代码与数据
    - 2.4.4 shell、父子进程与命令执行
- 3. 基础概念小结
  - 3.1 核心认识
  - 3.2 自检问题

本篇范围

> 按“计算机的数据流 -> 操作系统如何管理资源 -> Linux 如何表示和管理进程”的顺序展开。本篇只整理进程的定义、PCB/`task_struct`、进程的查看方式和 `fork()` 初识；进程状态、优先级、切换与调度、环境变量和地址空间等内容留到后续专题。

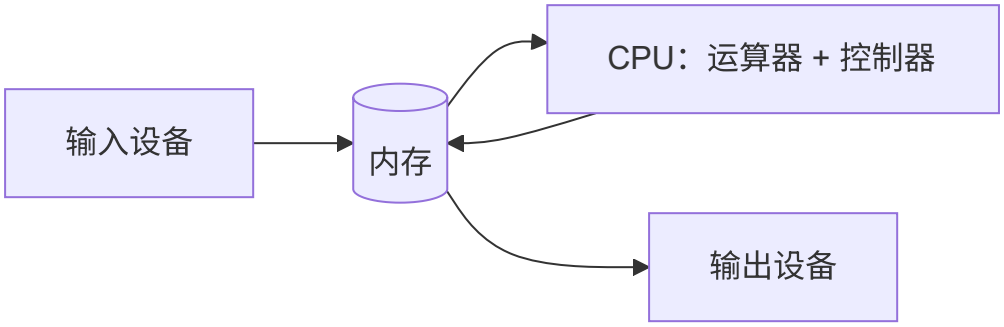
1. 从硬件系统到操作系统

1.1 冯·诺依曼体系结构

1.1.1 计算机的基本组成

常见计算机大多遵循冯·诺依曼体系结构。从功能上看，可以把它分成四部分：

1. **输入设备**：键盘、鼠标、扫描仪、网卡等，把外部信息送入计算机。
2. **存储器**：这里主要指内存，用来存放正在运行的程序及其数据。
3. **中央处理器（CPU）**：包括运算器和控制器，负责取指、运算和控制整个执行过程。
4. **输出设备**：显示器、打印机、网卡等，把处理结果送到外部世界。



这套结构真正要强调的，不只是“计算机由哪些零件组成”，而是这些零件之间**数据怎样流动**。

1.1.2 数据为什么必须先进入内存

在不单独讨论 CPU 缓存的简化模型中，数据层面的基本路径是：

```
输入设备 -> 内存 -> CPU -> 内存 -> 输出设备
```

CPU 计算时直接读写的是内存中的指令和数据；输入设备先把数据写入内存，CPU 再把内存中的数据取到寄存器中处理，处理结果先写回内存，最后才由输出设备读取并输出。

因此，冯·诺依曼体系可以概括为一句话：

> **在数据层面上，CPU 不直接和外设交换数据，所有设备都要通过内存参与数据流动。**

这里说的“输入”和“输出”，本质上也是站在**内存中的程序**这一侧观察问题：

- 从外部进入程序，就是输入；
- 从程序送到外部，就是输出。

一次聊天的数据流也可以这样拆解：键盘或网络把数据送入内存，CPU 对内存中的消息进行处理，处理结果再写回内存，最后由显示器和网卡读取并输出。发送文件虽然没有改变这套基本结构，但会让数据在内存、磁盘和网络设备之间发生多次拷贝。

数据经过的层次越多，通常需要拷贝的次数越多，效率也就越低。因此，程序的代码和数据必须先加载到内存中，CPU 才能在系统中调度它、执行它。这也是后面理解“进程”的来源。

1.1.3 从数据流过渡到进程

磁盘上的程序只是一个静态文件，它自身不会“运行”。要让 CPU 执行它，操作系统必须先把它的代码和相关数据装入内存，并为它建立一套运行状态信息。

当多个程序同时运行时，问题就出现了：内存中可能同时存在多套代码和数据，CPU 某一时刻又只能执行其中一套。操作系统必须知道：

- 现在有哪些程序正在运行；
- 每个程序执行到了哪里；
- 每个程序拥有哪些资源；
- 下一次应该让哪个程序使用 CPU。

这正是操作系统的进程管理工作。

1.2 操作系统

1.2.1 操作系统的概念

操作系统（Operating System，OS）可以理解为计算机系统中的一组基础软件。Linux 中通常包括：

- **内核**：负责进程管理、内存管理、文件管理和设备驱动管理等核心工作；
- **内核之外的程序与服务**：如系统服务、函数库、shell 等。

日常开发中经常用“操作系统”统称整个软件环境，但讲到资源管理时，实际承担核心工作的主体是**操作系统内核**。

1.2.2 对上提供环境，对下管理资源

操作系统处在硬件与应用之间，可以把它理解为“一个手段、一个目的”：

方向	主要职责
对下	与硬件交互，管理 CPU、内存、磁盘、网卡等软硬件资源
对上	为用户程序提供稳定、安全、高效的执行环境

如果没有操作系统，每个程序都要自己处理设备细节、内存分配和资源竞争，不仅开发困难，多个程序之间也容易互相破坏。操作系统把这些复杂工作统一接管起来，再向上提供相对统一的接口。

1.2.3 “管理”的本质：先描述，再组织

操作系统管理资源时，并不是直接对抽象概念发号施令，而是先把被管理对象变成内核可以处理的数据：

1. **先描述**：为被管理对象建立数据结构，记录它有哪些属性。
2. **再组织**：用链表、队列、树等数据结构把这些对象联系起来，方便查找、调度和修改。

例如，管理系统中的进程时，内核不会只保存一个“有进程在运行”的布尔值，而是会为每个进程建立一条详细记录，再把所有记录组织起来。

这个思想可以贯穿整个操作系统：**对象先被结构化描述，系统再组织这些结构。**

它也直接回答了后面的问题：

> Linux 怎样管理进程？先描述进程，再组织进程。

1.2.4 系统调用、库函数与 shell

用户程序通常不能随意访问内核数据和硬件资源，操作系统通过**系统调用**向上暴露受控接口。

系统调用偏底层，直接体现操作系统能力。库函数通常是对系统调用或底层功能的进一步封装，为开发者提供更方便的接口。需要注意：**库函数和系统调用不是同一个概念，也不是所有库函数都会发起系统调用。**

shell 可以理解为命令行解释器。我们输入一条命令时，shell 会解析命令，并通常通过系统调用创建子进程来执行它。常见的 `bash` 本身就是一个进程。

2. Linux 进程基础

2.1 从程序到进程

2.1.1 程序是什么

广义上，程序可以看作“逻辑 + 数据”的组合，也就是完成某件事的指令及其处理的数据。

编译完成后得到的可执行程序，在运行之前通常只是磁盘上的一个文件。它包含程序的指令和初始化数据，但还没有获得 CPU 执行权，也没有形成一套独立的运行时状态。

所以要先区分：

- **源程序**：程序员编写的源代码；
- **可执行文件**：经过编译、链接后放在磁盘上的文件；
- **进程**：可执行文件被操作系统加载并开始执行后形成的运行实体。

2.1.2 进程是什么

“进程”可以从三个角度理解：

角度	对进程的理解
课本角度	程序的一次执行实例，正在运行的程序
内核角度	承担资源分配的对象，可以获得 CPU 时间、内存等资源
Linux 实现角度	Linux 内核中的 `task_struct` 加上该程序自己的代码和数据

仅说“进程就是运行中的程序”容易遗漏关键部分：同样一个可执行文件可以被运行多次，形成多个彼此独立的进程。每个进程都有自己的 PID、运行状态、资源和执行位置。

2.1.3 进程与程序的核心关系

Linux 中可以把进程概括为：

```
进程 = 内核中的 task_struct + 程序自己的代码和数据
```

其中：

- `task_struct` 描述进程是谁、处于什么状态、执行到哪里、拥有哪些资源；
- 代码和数据是进程实际执行时所需要的内容。

这两个部分缺一不可。只有磁盘上的二进制文件，不能称为进程；只有内核中的管理结构而没有任何程序内容，也不能形成完整的用户进程。

因此，执行一条命令或启动一个应用，本质上是让操作系统创建并运行一个进程。例如：

```
gcc hello.c -o hello
```

执行这条命令时，`gcc` 本身也是由 shell 创建出来的进程；它的任务是把源文件编译、链接为可执行文件。生成的 `hello` 是磁盘文件，只有再次被执行时，才会变成一个新的进程。

2.2 进程控制块 PCB 与 task_struct

2.2.1 PCB：描述进程的数据结构

操作系统为了管理进程，会为每个进程建立一块保存进程属性的数据结构。这个结构通常称为**进程控制块（Process Control Block，PCB）**。

PCB 可以理解为“进程属性记录的集合”。没有它，内核就无法稳定地回答这些问题：

- 这个进程的唯一身份是什么？
- 它现在能否运行？
- 它上一次执行到了哪里？

- 它占用了哪些内存和文件？
- 是否应该让它获得 CPU？

所以 PCB 不只是一个附加标签，而是操作系统能够识别、调度和管理进程的基础。

2.2.2 task_struct：Linux 的 PCB

PCB 是一个通用概念，不同操作系统有不同实现。Linux 用 `task_struct` 结构体描述进程，因此：

```
PCB 是一类数据结构；“task_struct” 是 Linux 中的具体实现。
```

`task_struct` 会存在于内核内存中，包含进程运行所需的各种管理信息。

2.2.3 task_struct 中的基础属性

不同内核版本的字段会变化，但从功能上可以先认识以下八类信息：

1. **标识符**：例如 PID、PPID，用来识别当前进程以及它与父进程的关系。
2. **状态**：记录进程正在运行、等待、停止等状态，也保存退出代码、退出信号等信息。
3. **优先级**：决定进程竞争 CPU 时的相对顺序。优先级的背后是有限的 CPU 资源。
4. **程序计数器**：保存下一条待执行指令的位置，使进程从 CPU 上暂时离开后仍能继续执行。
5. **内存指针**：描述程序代码、数据以及共享内存所在的位置。
6. **上下文数据**：保存进程离开 CPU 时的寄存器等重要运行现场，恢复后可以继续执行。
7. **I/O 状态信息**：记录 I/O 请求、分配到的 I/O 设备以及正在使用的文件等信息。
8. **记账信息**：记录 CPU 使用时间、资源使用量、时间限制等，用于统计和限制。

CPU 寄存器只有一套，但内核可以保存多个进程各自的运行现场。进程切换时，当前进程的现场被保存，下一个进程的现场被恢复；详细机制放到后续“进程切换与调度”专题。

2.2.4 内核如何组织进程

系统中会同时存在大量 `task_struct`。内核不会让它们彼此孤立，而是通过链表、队列或其他数据结构将它们组织起来。

这就是“先描述、再组织”的具体落地：

```
描述一个进程 -> task_struct
组织所有进程 -> 将 task_struct 放入内核管理结构
```

当用户查看进程、内核进行调度或进程发生状态变化时，本质上都在访问这些被组织起来的内核数据结构。

2.3 查看进程

2.3.1 通过 /proc 查看进程

Linux 的 `/proc` 是一个虚拟文件系统，用来向用户展示内核和进程的运行时信息。每个进程通常对应一个以 PID 命名的目录：

```
/proc/<PID>/
```

例如查看当前 shell 的进程信息和工作目录：

```
ls -l /proc/$$
readlink /proc/$$/cwd
```

其中 `$$` 表示当前 shell 的 PID。进程存在时，`/proc/<PID>` 目录存在；进程结束后，对应目录也会消失。

这种“用文件形式展示进程属性”的方式，方便用户和程序读取内核状态，但其中的文件通常不是磁盘上的普通文件，而是内核实时生成的视图。

2.3.2 通过 ps 和 top 查看进程

`ps` 适合查看某一时刻的进程快照，`top` 适合持续观察系统中进程的资源使用情况。

查看当前 shell 的 PID、PPID、状态和命令名：

```
ps -p $$ -o pid,ppid,stat,comm
```

查看某一进程时，也可以先打印 `ps axj` 的标题，再用进程名筛选：

```
ps axj | head -n 1
ps axj | grep hello
```

PID 是进程在当前系统中的标识。PID 通常按一定规则分配，并可能在达到范围上限后回绕；系统中的其他任务也在持续使用 PID，因此不能把“PID 是否连续”当作判断进程关系的依据。

2.3.3 通过系统调用获取 PID 和 PPID

进程可以通过系统调用读取自己的标识信息：

- `getpid()`：获取当前进程的 PID；
- `getppid()`：获取当前进程的父进程 PID。

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    printf("pid  = %d\n", getpid());
    printf("ppid = %d\n", getppid());
    return 0;
}
```

运行程序后，可以将输出与 `ps` 显示的结果相互验证：程序自己看到的 PID，应当就是操作系统为它分配的进程标识。

2.4 创建进程：fork 初识

2.4.1 fork 做了什么

Linux 通常不是凭空创建一个完整的用户进程，而是让已有进程通过 `fork()` 系统调用创建子进程。

调用 `fork()` 后，内核会为子进程建立新的进程管理信息，并让父子进程从 `fork()` 之后的位置分别继续执行。可以把它概括为：

> **调用一次，返回两次；调用前是一个执行流，调用后分成父子两个执行流。**

2.4.2 返回值与父子进程分流

`fork()` 的返回值是区分父子进程的关键：

返回值	含义
小于 `0`	创建子进程失败
等于 `0`	当前代码运行在子进程中
大于 `0`	当前代码运行在父进程中，返回值是新子进程的 PID

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    pid_t id = fork();

    if (id < 0) {
        perror("fork");
        return 1;
    } else if (id == 0) {
        printf("child : pid=%d, ppid=%d\n", getpid(), getppid());
    } else {
        printf("parent: pid=%d, child=%d\n", getpid(), id);
    }

    return 0;
}
```

为什么这样设计？

- 父进程可能创建多个子进程，因此需要拿到每个子进程的 PID 才能分别管理；
- 子进程只需先用返回值 `0` 判断“我是子进程”，需要父进程信息时再调用 `getppid()`。

因此，`fork()` 后通常要用 `if` 或 `switch` 按返回值分流。`id == 0` 不是某个进程的 PID，只是给子进程的返回标记。

2.4.3 父子进程的代码与数据

父子进程会继续执行相同的程序代码，但从 `fork()` 开始形成两个独立的执行流。对于数据，可以用下面的基本认识理解：

- 读操作通常可以共享同一份物理数据；
- 任意一方准备写入时，内核再为该进程准备独立的副本。

这种机制称为**写时拷贝（Copy-on-Write，COW）**。它避免 `fork()` 时立刻复制全部内存，提高了创建进程的效率。

“为什么同一个变量地址在父子进程中相同、值却可以不同”涉及虚拟地址空间与页表，留到地址空间专题，不在基础概念中展开。

2.4.4 shell、父子进程与命令执行

在交互式 shell 中执行命令时，常见过程是：

```
bash 创建子进程 -> 子进程执行目标程序 -> bash 等待或继续接受命令
```

所以，我们运行的程序通常把 `bash` 显示为父进程。执行命令的本质不是“让 shell 自己变成目标程序”，而是让 shell 创建或控制新的进程来完成工作。

这也把前面几部分连接了起来：

```
用户在 shell 输入命令
        ↓
shell 通过系统调用创建进程
        ↓
内核建立新的 task_struct，并准备代码和数据
        ↓
新进程进入系统，等待或获得 CPU 执行
```

3. 基础概念小结

3.1 核心认识

Linux 进程基础概念可以串成一条主线：

1. 程序执行时，代码和数据必须先进入内存，CPU 才能处理它们。

2. 操作系统对下管理资源，对上提供执行环境。
3. 管理的基本方法是“先描述，再组织”。
4. Linux 用 `task_struct` 描述进程，再把它放入内核管理结构。
5. 进程 = `task_struct` + 自己的代码和数据。
6. 可以通过 `/proc`、`ps`、`top` 以及 `getpid()`、`getppid()` 查看进程。
7. `fork()` 从已有进程创建子进程，是理解 Linux 进程创建方式的起点。

3.2 自检问题

学完基础概念后，应能直接回答

1. 为什么程序必须先加载到内存中，CPU 才能执行？
2. “程序”“可执行文件”和“进程”之间有什么区别？
3. 为什么 Linux 需要 `task_struct`，它和 PCB 是什么关系？
4. `fork()` 为什么会有不同的返回值，父子进程如何分流？
5. `/proc/<PID>`、`ps` 和 `getpid()` 分别从什么角度获取进程信息？

后续专题

进程状态、僵尸进程与孤儿进程见 2026-7-12 Linux进程--状态；进程切换和调度见 2026-7-12 Linux进程--切换和调度；命令行参数见 2026-7-14 Linux进程--命令行参数；环境变量见 2026-7-14 Linux进程--环境变量。